

評估三 A: 設計對抗樣本 (實驗結果文檔)

深度學習模型訓練與測試準確率

本實驗採用 FashionMNIST 資料集 (包含 60,000 張訓練圖像與 10,000 張測試圖像)，訓練了一個卷積神經網路 (CNN) 分類器。模型訓練參數設定如下：

- 訓練輪數 (Epochs): 15
- 批次大小 (Batch Size): 64
- 學習率 (Learning Rate): 0.001

接下来为模型训练声明若干超参数 (hyperparameters)，包括：训练轮数 (training epochs)、批次大小 (batch size) 与学习率 (learning rate)。

训练轮数可以设为大于 10 的任意整数；批大小可设为 32、64 或 100；学习率设为 0.001。

```
# 聲明超參數
epochs = 15      # 訓練輪數
batch_size = 64  # 批次大小
learning_rate = 0.001 # 學習率
```

3) ✓ 0.0s

Python

加载训练集与测试集。所需数据集名称为 **FashionMNIST**。获取数据集的方式与实践课一样，只需将 **'MNIST'** 替换为 **'FashionMNIST'**。

```
# 載入 FashionMNIST 訓練集與測試集
train_dataset = torchvision.datasets.FashionMNIST(root='./data', train=True, transform=transforms.
ToTensor(), download=True)
test_dataset = torchvision.datasets.FashionMNIST(root='./data', train=False, transform=transforms.
ToTensor(), download=True)
# 使用 DataLoader 將數據集分批載入
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size, shuffle=False)
```

5) ✓ 0.0s

Python

请分别输出训练集与测试集的样本数量。

期望输出：训练集的样本数量：60000 测试集的样本数量：10000

```
# 輸出樣本數量
print(f"訓練集的樣本數量：{len(train_dataset)}")
print(f"測試集的樣本數量：{len(test_dataset)}")
```

6) ✓ 0.0s

Python

```
訓練集的樣本數量： 60000
測試集的樣本數量： 10000
```

經過訓練和 10,000 張測試圖像驗證後，模型展現出優異的分類能力。

- 整體測試準確率：92.13%
- 個別服飾類別中辨識率最高：
 - Trouser (99.10%)
 - Sneaker (98.80%)
 - Bag (98.50%)

模型在 10000 張測試圖像上的測試準確率： 92.13 %

模型在類別 'T-shirt/top'	上的測試準確率： 86.70 %
模型在類別 'Trouser'	上的測試準確率： 99.10 %
模型在類別 'Pullover'	上的測試準確率： 85.70 %
模型在類別 'Dress'	上的測試準確率： 93.20 %
模型在類別 'Coat'	上的測試準確率： 88.40 %
模型在類別 'Sandal'	上的測試準確率： 98.00 %
模型在類別 'Shirt'	上的測試準確率： 78.30 %
模型在類別 'Sneaker'	上的測試準確率： 98.80 %
模型在類別 'Bag'	上的測試準確率： 98.50 %
模型在類別 'Ankle boot'	上的測試準確率： 94.60 %

為了準備生成對抗樣本，從 FashionMNIST 訓練集中提取了前 5 張圖片。

- 前 5 張圖片真實標籤依序為：
 1. Ankle boot
 2. T-shirt/top
 3. T-shirt/top
 4. Dress
 5. T-shirt/top



在未加入任何干擾噪聲之前，將第一張圖片 (Ankle boot) 輸入訓練好的 CNN 模型進行預測。模型成功且正確地輸出預測標籤「Ankle boot」，證明模型在正常情況下能夠準確識別這些圖像。

```
# 註：官方 FashionMNIST 訓練集的第一張圖 (index 0) 真實標籤為 Ankle boot。
# 若期望輸出為 Trouser，應測試 index 16 的圖像。這裡依題目要求測試第一張圖。
model.eval()
with torch.no_grad():
    img = train_dataset[0][0].unsqueeze(0).to(device)
    output = model(img)
    _, predicted_label = torch.max(output.data, 1)
    print(f"預測標籤: {classes[predicted_label.item()]}")
```

16] ✓ 0.0s

預測標籤: Ankle boot

本階段實施了規避攻擊(**Evasion Attack**)，目標是在不改變人類肉眼視覺辨識的前提下，誘導機器學習模型做出錯誤分類。

對抗樣本生成參數：

- 生成了一個與原始圖像尺寸相同(28x28)的隨機噪聲矩陣。
- 噪聲縮放比例 (**noise_scale**): 1.9(將隨機噪聲除以 1.9 後疊加至原圖上)。
- 使用 **np.clip** 確保像素值保持在合理的 0~1 範圍內，避免圖片過度失真。

攻擊結果：

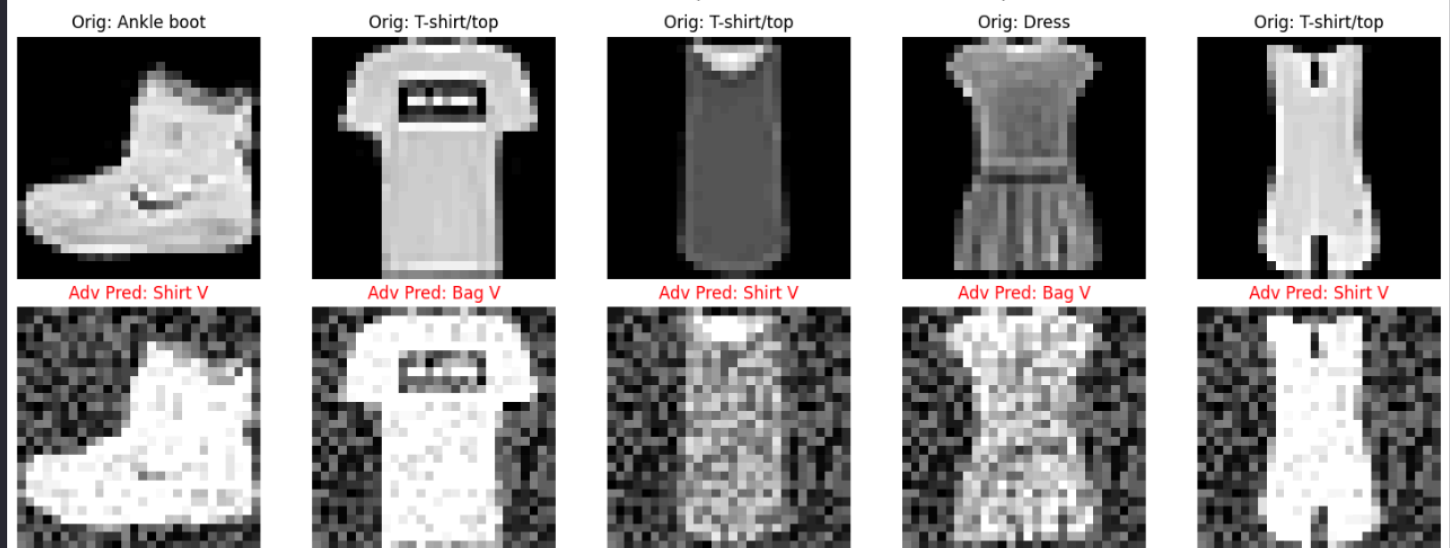
如下圖對比所示，對抗樣本(下排)在視覺上與原圖(上排)高度相似，仍可清楚辨識出鞋子與衣服的輪廓。然而，微小的像素干擾成功欺騙了 CNN 分類器，**5 張對抗樣本的預測結果全數錯誤**(攻擊成功率 **100%**)。

預測結果變化總結：

- 圖片 1：原標籤 [Ankle boot] → 誤判為 [Shirt] (Success)
- 圖片 2：原標籤 [T-shirt/top] → 誤判為 [Bag] (Success)
- 圖片 3：原標籤 [T-shirt/top] → 誤判為 [Shirt] (Success)
- 圖片 4：原標籤 [Dress] → 誤判為 [Bag] (Success)
- 圖片 5：原標籤 [T-shirt/top] → 誤判為 [Shirt] (Success)

----- 目前測試的雜訊縮放比例為：1/1.9 -----

Random Noise Attack (Noise scaled down to 1/1.9)



=== 預測結果總結 ===

```

圖片 1：原標籤 [ Ankle boot] ---> 預測為 [      Shirt] (Success 成功欺騙!)
圖片 2：原標籤 [ T-shirt/top] ---> 預測為 [      Bag] (Success 成功欺騙!)
圖片 3：原標籤 [ T-shirt/top] ---> 預測為 [      Shirt] (Success 成功欺騙!)
圖片 4：原標籤 [      Dress] ---> 預測為 [      Bag] (Success 成功欺騙!)
圖片 5：原標籤 [ T-shirt/top] ---> 預測為 [      Shirt] (Success 成功欺騙!)
  
```